



量子线路路由编译实验

量子信息基础大作业

学号 3230105807

姓名 李浩瑄

课程名称 量子信息基础

日期 2026 年 6 月

指导老师 钱浩亮

目录

摘要	1
1 引言	1
1.1 实验背景	1
1.2 实验目标	2
1.3 实验设计思路	2
2 实验环境与问题建模	3
2.1 量子线路基础	3
2.2 线路与硬件拓扑	3
2.3 映射与路由目标	5
2.4 课程知识对应	6
3 SABRE 基线复现实验	6
3.1 算法思路	6
3.2 任务定义	7
3.3 复现设置	7
3.4 结果分析	8
4 NPQR 路由算法设计	8
4.1 路由闭环	9
4.2 前沿门推进	10
4.3 初始映射	11
4.4 搜索策略	11
4.5 动作评分	12
4.6 局部修复	13
4.7 复放验证	13
4.8 算法过程	14
5 实验系统设计	14
5.1 演示界面	15
5.2 调用方式	15
5.3 拓扑与样例	15
5.4 轨迹回放	16
5.5 功能结构	16
6 项目部署与交付	16
6.1 项目组织	17
6.2 在线前端	18
6.3 服务器部署	18
6.4 工具接口	18
6.5 测试与复现	18

7 AI 辅助开发	19
7.1 资料整理	19
7.2 网页开发	19
7.3 服务开发	20
7.4 部署测试	20
7.5 报告整理	20
7.6 过程控制	21
8 实验结果与数据分析	21
8.1 实验设置	21
8.2 基础实验	22
8.3 拓展实验	23
8.4 消融实验	23
8.5 运行过程	24
9 实验讨论	25
9.1 复杂度	25
9.2 方法对比	27
9.3 结果解释	27
10 总结与体会	27
参考资料	28
A 附录：复现实验记录	28
A.1 在线地址	29
A.2 本地安装	29
A.3 REST API	29
A.4 MCP 服务	29
A.5 示例编译	29
A.6 测试命令	29

摘要

量子线路模型把量子计算表示为一串酉操作，但真实量子芯片并不支持任意两个物理量子比特直接执行双比特门。逻辑线路进入硬件之前，编译器必须根据耦合拓扑安排逻辑量子比特的位置，并在必要时插入 SWAP，使每个 CNOT 或 CZ 都落在可执行的物理边上。SWAP 保持理想量子语义，却会增加门数、线路深度和噪声暴露时间，因此量子线路路由是量子信息基础概念走向真实芯片执行的关键环节。本项目围绕这一问题实现了一个学习评分量子线路路由编译系统。报告先复现经典 SABRE 路由思想，再在同一实验口径下构建 NPQR 方法；该方法把初始映射、动作掩码、模型评分、束搜索、局部修复和轨迹复放组合成一个可解释的搜索流程，用于在受限拓扑上生成语义等价的可执行线路。

作为量子信息基础大作业，本项目按完整系统项目形式完成。项目从基线复现开始，经历算法迭代、系统封装和公开交付，形成了 GitHub 开源仓库、README、测试用例、在线前端、服务器端 REST 服务、MCP/命令行接口和报告文档。GitHub Pages 前端用于展示线路输入、拓扑选择、映射变化、轨迹回放和指标对比；云端服务负责执行真实编译请求；本地测试覆盖接口契约、页面契约、示例数据和交付验证。AI 工具在项目中承担工程辅助角色，主要用于论文与资料梳理、前后端开发提示、测试用例整理、部署验证和报告文字修订；算法目标、实验口径和结果解释由作者根据代码运行结果确定。上述材料共同覆盖课程要求中的理论理解、算法实现、实验验证、过程记录、在线展示和可复现交付。

实验围绕 SABRE basic 与 NPQR 的对照展开。10/20 比特代表线路覆盖 GHZ、QFT、QAOA、VQE、随机线路和结构化线路，NPQR 在 10 个样例上全部完成路由，SWAP 数均低于 SABRE basic；例如 QAOA10 从基线的 15 个 SWAP 降至 0 个，DeepRandom20 从 135 个降至 105 个。选定 30/50 比特网格样例用于观察更大拓扑下的扩展效果，4 个样例全部完成，并在 SWAP 数上保持优势。阶段耗时分析显示，不同线路的瓶颈并不相同：QAOA10 和 Brickwork20 的主要时间消耗在初始映射候选生成，QFT10 的主搜索平均耗时 14.65 秒，占总耗时 90.2%，其中模型评分和束搜索是主要子开销。整体结果表明，本项目已经形成从量子信息建模、SABRE 复现、自研路由方法、实验对照到网页与服务部署的闭环。

1 引言

1.1 实验背景

量子计算的线路模型给出了一个清晰的抽象：量子寄存器保存量子态，量子门按时间顺序作用在寄存器上，整个计算过程对应复向量空间中的酉演化。课程中讨论的叠加、纠缠、测量、CNOT、QFT 和 QAOA 等概念，都可以在线路模型中得到统一描述。这个模型适合讲清楚量子信息的数学结构，也适合作为算法设计的起点。

真实芯片给这个模型增加了硬件约束。以超导量子芯片为例，物理量子比特按固定

拓扑连接，双比特门只能在相邻物理量子比特之间直接执行。理想线路中的两个逻辑量子比特可能在算法层面相互作用频繁，但映射到芯片后，它们未必位于相邻物理位置。编译器需要在逻辑线路和物理拓扑之间建立对应关系，并在不相邻时插入 SWAP 重新安排量子态。这一步不会改变理想语义，却会增加额外门操作。在线路深度增大后，退相干、门误差和串扰带来的影响更明显，路由质量会直接影响量子程序在真实设备上的可执行性和可靠性。

量子线路路由因此成为一个典型的交叉问题。一方面，它建立在量子信息基础之上：每个门都是对量子态的西变换，SWAP 对应两个量子比特状态的交换，硬件拓扑限制了双比特相互作用的物理位置。另一方面，它又是一个组合优化问题：编译器必须在大量可能映射和候选 SWAP 中选择一条路径，使输出线路满足拓扑约束，同时尽量减少 SWAP 数、深度和编译耗时。对于几十个量子比特的线路，简单枚举不可行，启发式搜索和学习辅助方法具有实际意义。

1.2 实验目标

本项目从复现 SABRE 基线开始。SABRE 是量子路由中常用的启发式方法，它利用前沿门、物理距离和后续门信息选择候选 SWAP，能够在较短时间内给出稳定结果。先实现并复现这一基线，有两个作用：第一，它给出了传统方法的可比较参照；第二，它让后续自研方法始终在同一线路、同一拓扑和同一指标下讨论，而不是只展示单独算法的绝对数值。

在此基础上，项目设计了 NPQR 学习评分路由方法。NPQR 并不把路由完全交给学习模型决策，而是把模型放入显式搜索流程中：初始映射根据逻辑交互结构选择起点，动作掩码缩小候选 SWAP 集合，模型评分给候选动作排序，束搜索保留多条可能路线，局部修复处理尾部卡顿状态，轨迹复放验证每一步门是否满足硬件边约束。这样的设计保留了搜索过程的可解释性，也使结果能够通过可视化回放和指标表格相互印证。

项目最终不只停留在算法脚本层面，而是实现为一个可演示、可运行、可复现的系统成果。系统支持网页演示、服务器接口、工具接口和本地命令行；前端展示线路输入、拓扑映射、SWAP 插入和指标对比；后端负责执行编译、返回结果并支持扩展实验。围绕这些交付形式，项目补齐 README、测试、部署说明和报告材料，使代码实现、在线页面、服务接口和本地示例形成统一交付。

1.3 实验设计思路

本文按“基础问题、基线复现、自研方法、系统交付、实验验证”的顺序展开。后文先介绍量子比特、量子门、SWAP、硬件拓扑和路由目标，再说明 SABRE 基线的复现方法与实验口径；随后给出 NPQR 的自然演进过程，并介绍系统界面、轨迹回放、部署方式和测试验证；在过程说明部分，报告单独讨论 AI 协作在资料整理、开发、测试和写作中的使用方法；实验部分集中呈现主实验、扩展实验、消融实验和耗时分析；最后总结课程知识、算法实现、工程交付和后续优化方向。

2 实验环境与问题建模

本章把报告后续使用的量子信息概念和路由编译模型放在同一条线上说明。量子线路路由不是普通图搜索的直接套用：算法每插入一个 SWAP，都对应量子态在物理载体上的交换；算法每推进一个双比特门，都必须同时满足线路依赖关系和芯片耦合约束。把这一层关系讲清楚，后面的 SABRE 复现、NPQR 设计和实验指标才有明确的物理含义。

2.1 量子线路基础

一个量子比特的纯态写作

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1.$$

这里 $|0\rangle$ 和 $|1\rangle$ 是计算基， α 、 β 是复振幅。测量得到 0 或 1 的概率分别为 $|\alpha|^2$ 和 $|\beta|^2$ 。多个量子比特组成寄存器后，状态空间由张量积给出， n 个量子比特对应 2^n 维复向量空间。量子线路中的门操作不是任意线性变换，而是保持内积和概率归一性的酉变换。

这一定义决定了路由编译的基本边界。编译器可以改变逻辑量子比特放置在哪个物理位置，也可以插入语义明确的辅助门，但不能改变原线路描述的量子演化。换言之，路由输出的线路必须和输入线路在逻辑语义上等价，只是执行时使用了不同的物理量子比特顺序和额外的交换操作。

量子门与 SWAP

单比特门作用在一个二维子空间上，常见例子包括 X 、 H 和相位门。双比特门作用在两个量子比特的张量积空间上，是产生和调控纠缠的关键。本项目关注的硬件约束主要来自双比特门，因为真实芯片通常只允许相邻物理量子比特之间直接执行 CNOT 或 CZ。

SWAP 是路由编译中最重要的辅助门。它的作用可以写为

$$\text{SWAP}|a\rangle|b\rangle = |b\rangle|a\rangle.$$

在矩阵形式下，SWAP 是酉矩阵；在物理解释上，它交换两个物理位置上承载的逻辑量子态。因此，若逻辑量子比特 q_i 和 q_j 当前不相邻，编译器可以沿硬件边插入若干 SWAP，把二者移动到相邻位置后再执行目标双比特门。这个过程保持理想线路语义，却增加门数和深度。

2.2 线路与硬件拓扑

量子线路可以看成有序门列

$$C = (g_1, g_2, \dots, g_m).$$

单比特门只依赖其作用的一个逻辑量子比特；双比特门依赖两个逻辑量子比特的当前状态。如果两个门作用在同一逻辑量子比特上，后一个门必须等待前一个门完成。这些先后关系构成线路依赖结构。路由算法中的“前沿门”正是依赖已经满足、下一步可以尝试执行的门。

线路结构影响路由难度。GHZ 线路通常形成链式或星形纠缠，QFT 线路包含密集的长距离双比特交互，QAOA 和 VQE 线路则由问题图或变分层决定交互模式。若交互图和硬件拓扑形状接近，路由器可以少插入 SWAP；若逻辑交互跨越拓扑远端，编译器必须频繁移动量子态。后续实验选择这些线路类型，就是为了覆盖不同交互结构下的路由行为。

硬件拓扑

真实芯片的物理连接用图表示：

$$G = (V, E),$$

其中 V 是物理量子比特集合， E 是可直接执行双比特门的耦合边。若 $(u, v) \in E$ ，映射到 u 和 v 上的两个逻辑量子比特可以直接执行双比特门；若 $(u, v) \notin E$ ，编译器必须先通过 SWAP 改变映射。

图 1 给出本报告使用的两类拓扑示意。20 比特代表实验使用 Tokyo 风格耦合图，扩展实验使用二维网格。两类拓扑的共同点是连接稀疏：每个物理量子比特只和少数邻居相连，远距离逻辑交互必须通过多次交换完成。系统会预先计算物理距离 $d_G(u, v)$ ，即两个物理量子比特在耦合图中的最短路径长度，并把它作为候选 SWAP 评分、前沿门代价和初始映射选择的重要依据：

$$d_G(u, v) = \text{dist}_G(u, v).$$

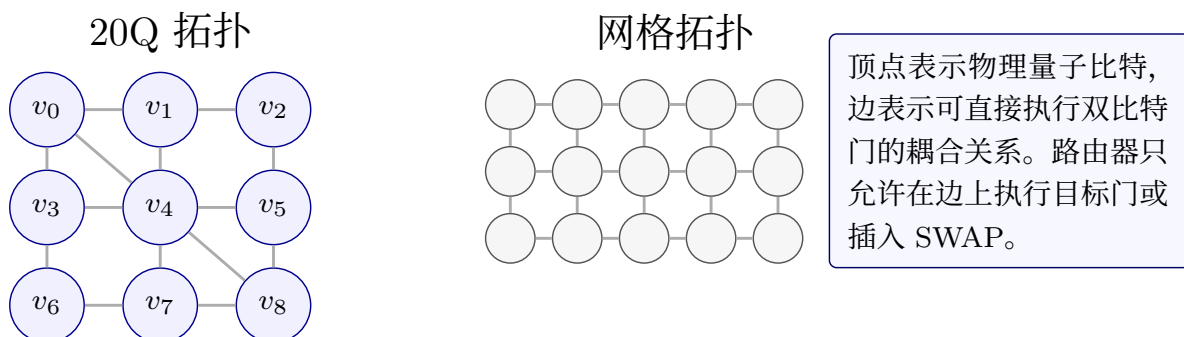


图 1: 硬件拓扑

2.3 映射与路由目标

逻辑量子比特集合记为 Q 。路由过程中的映射写作

$$\pi : Q \rightarrow V,$$

表示逻辑量子比特当前被放置到哪个物理位置。若双比特门 $g = (q_i, q_j)$ 满足

$$(\pi(q_i), \pi(q_j)) \in E,$$

该门可以在当前映射下直接执行。若条件不满足，路由器需要选择一条物理边 (u, v) 插入 SWAP，并交换这两个物理位置上承载的逻辑量子比特。设 π' 是执行 SWAP 后的新映射，则位于 u 和 v 的两个逻辑量子比特互换位置，其余逻辑量子比特保持不变。

图 2 用一个小例子说明映射和 SWAP 的关系。逻辑线路中 q_0 和 q_2 需要执行 CNOT，但初始映射把它们放在不相邻的物理位置。编译器不能直接执行该门，只能先在硬件边上插入 SWAP，让逻辑量子态沿拓扑移动。这个例子解释了为什么路由不是简单地把每个逻辑门翻译成一个物理门：一次 SWAP 会改变后续所有门看到的物理位置，当前选择和未来代价紧密耦合。

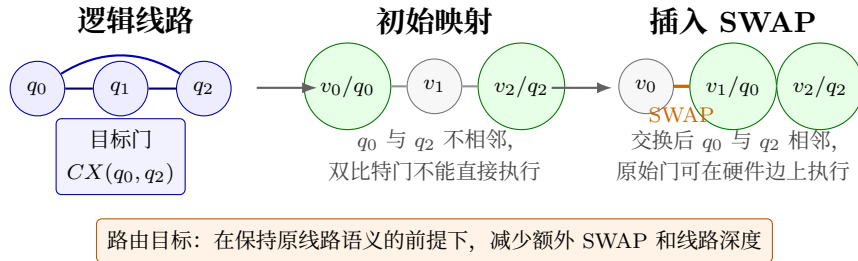


图 2: 路由问题

评价指标

本项目使用三个主要指标衡量路由结果：SWAP 数、线路深度和编译耗时。SWAP 数表示为满足拓扑约束额外插入了多少交换门，它直接反映路由带来的额外操作。线路深度表示按依赖关系分层后需要多少时间步执行，深度越大，量子态在噪声环境中停留的时间越长。编译耗时表示算法生成结果需要的时间，它决定在线演示、服务器接口和本地工具的响应速度。

三者可以写成一个综合优化目标：

$$\min \alpha N_{\text{swap}} + \beta D + \gamma T,$$

其中 N_{swap} 是插入 SWAP 数， D 是路由后线路深度， T 是编译耗时， α, β, γ 用于表示不同指标的权重。实际系统不求解精确全局最优解，而是在有限时间内寻找拓扑合法、语义等价、质量较好的可行线路。这个选择符合量子信息基础大作业的实验目标：重点比

较不同算法设计在相同线路和拓扑下的可执行结果，而不是把问题化为一个难以在报告规模内完整求解的精确优化模型。

2.4 课程知识对应

表 1 概括了课程知识和本项目实现之间的对应关系。该表的作用不是把课程概念逐条罗列，而是说明路由编译为什么适合作为量子信息基础大作业主题：它同时涉及量子态、西门、纠缠线路、硬件约束、噪声影响和近似优化。

表 1: 课程对应

知识点	项目体现
量子态	线路中的门序列描述量子态演化，路由输出必须保持逻辑语义等价。
张量积	多量子比特寄存器位于张量积空间，双比特门作用在两个子系统上。
西门	CNOT、CZ、SWAP 都是西门；额外 SWAP 改变承载位置，不改变理想演化。
纠缠	GHZ、QFT 等样例包含密集双比特交互，对硬件拓扑更敏感。
拓扑	芯片耦合关系被建模为图，只有图边上的物理量子比特能直接执行双比特门。
噪声	额外 SWAP 和更深线路会增加噪声暴露时间，减少交换门具有物理意义。
优化	映射选择、候选 SWAP、搜索剪枝构成受约束的组合优化过程。
验证	轨迹回放逐步检查门操作和映射变化，确认输出线路满足拓扑约束。

3 SABRE 基线复现实验

自研路由算法之前，报告先复现 SABRE。这样安排有两个原因：一是 SABRE 本身就是量子线路映射领域常用的启发式方法，适合作为量子信息基础大作业的经典算法起点；二是后续 NPQR 的效果需要放在同一批线路、同一类拓扑和同一组指标下讨论。没有稳定基线，单独报告一个 SWAP 数很难说明算法质量。

3.1 算法思路

SABRE 的核心思想是围绕“前沿门”逐步改善映射。在线路依赖关系中，所有前驱已经完成的门构成当前前沿。若前沿中的双比特门已经落在硬件边上，路由器直接执行该门；若它的两个逻辑量子比特在当前映射下不相邻，路由器就在相关物理邻边中选择一个 SWAP，更新映射后继续检查前沿。

这个过程把路由问题转化为局部搜索。每一步不尝试枚举完整线路的所有映射序列，而是用当前前沿门和少量后续门估计候选 SWAP 的价值。设前沿门集合为 F ，当前映

射为 π ，硬件图距离为 d_G ，一个常用的前沿距离代价可以写成

$$H_{\text{front}}(\pi) = \sum_{(q_i, q_j) \in F} d_G(\pi(q_i), \pi(q_j)).$$

若候选 SWAP 使这个代价下降，前沿门离可执行状态就更近。SABRE 的 lookahead 和 decay 变体会进一步考虑后续门或重复移动惩罚；basic 变体保留最直接的距离启发式，适合作为固定、可复现的质量基线。

3.2 任务定义

候选 SWAP 不需要从全部物理边中盲目选择。SABRE 通常先找出与前沿门相关的逻辑量子比特，再把这些逻辑量子比特当前所在物理位置的邻边作为候选集合。这样做减少了搜索分支，也把每一步选择集中在最可能推进当前线路的位置。

候选动作的评价具有明显的“当前与未来”权衡。只看当前门时，最短路径上的某个 SWAP 往往很有吸引力；但这个 SWAP 可能把另一个即将参与后续门的逻辑量子比特移远。SABRE 的启发式价值在于，它用前沿门和扩展集合共同估计一个动作，而不是只为当前门走一步最短路。这也是本项目后续设计自研方法时保留“前沿门”“候选动作”“物理距离”这些概念的原因。

3.3 复现设置

本项目复现 SABRE 时采用固定口径。输入线路使用 OpenQASM 2 表示，硬件拓扑使用 Tokyo 风格耦合图；映射预处理先把逻辑量子比特放入目标拓扑，再执行 SABRE 路由。路由阶段比较 basic、lookahead 和 decay 三种启发式，随机种子固定为 42，单次基线使用 1 次 trial；输出统一统计完成状态、SWAP 数和线路深度。后续主实验中的“SABRE basic”即采用这一固定口径。

表 2 给出小规模复现结果。GHZ5 在三种启发式下均不需要额外 SWAP，说明其交互结构已经适配当前拓扑。QFT5 和 QFT10 的长距离双比特交互更多，三种启发式之间出现差异；在 QFT10 上，lookahead 的 SWAP 数低于 basic，说明后续门信息能改善局部选择。报告后续仍选 basic 作为主要对照，是因为它定义清晰、参数少，更适合作为固定基线；lookahead 和 decay 则用于说明 SABRE 本身也存在启发式设计空间。

表 2: SABRE 结果

线路	启发式	SWAP 数	深度	状态
GHZ5	basic	0	7	是
GHZ5	lookahead	0	7	是
GHZ5	decay	0	7	是
QFT5	basic	8	53	是
QFT5	lookahead	7	55	是
QFT5	decay	7	55	是
QFT10	basic	42	168	是
QFT10	lookahead	29	156	是
QFT10	decay	30	167	是

3.4 结果分析

SABRE 复现给后续工作提供了三层参照。第一，它验证了线路解析、拓扑建模、映射更新和 SWAP 统计这条基本实验链路可以运行。第二，它提供了传统启发式的质量水平，使 NPQR 的 SWAP 数、深度和耗时可以在同一指标下比较。第三，它暴露了经典启发式的局部性：当线路包含密集长距离交互时，单步启发式很容易受到当前前沿门支配。后续方法章围绕初始映射、束搜索和模型评分展开，正是为了在保留 SABRE 可解释结构的同时扩大搜索视野。

4 NPQR 路由算法设计

NPQR 是本项目的自研路由方法。它的出发点不是用学习模型替代整个编译器，而是把学习得到的动作偏好放进一个显式搜索框架中。这个框架始终维护逻辑到物理量子比特的映射，按线路依赖推进前沿门，只在硬件边上插入 SWAP，并用轨迹复放验证输出线路。因此，NPQR 的核心是一条可解释的路由流程：搜索负责保证约束和状态更新，模型负责帮助候选动作排序。

图 3 给出方法流程。输入线路和硬件拓扑先被转化为依赖关系、距离矩阵和逻辑交互结构；初始映射给搜索提供起点；前沿门推进、动作生成、评分、搜索和修复共同产生路由轨迹；最终结果通过复放后输出线路和指标。

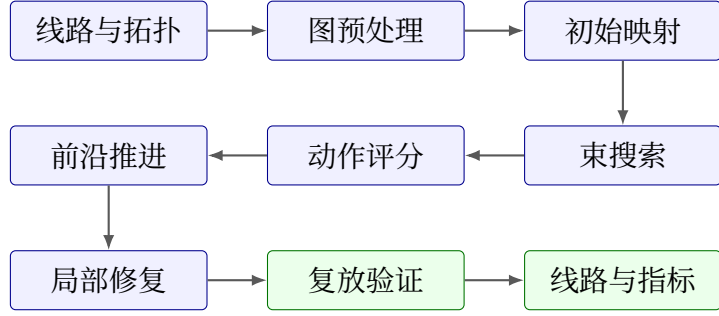


图 3: 方法流程

表 3 概括了方法从基础闭环到学习评分的自然演进。这个顺序也反映了项目开发中的真实技术路线：先把线路依赖、映射更新和合法性验证打通，再逐步把初始布局、搜索宽度和学习评分纳入同一个路由框架。

表 3: 阶段变化

阶段	主要变化	解决的问题
路由闭环	建立线路依赖、映射维护、SWAP 插入和轨迹复放	先保证输出线路拓扑合法、语义一致。
初始映射	根据逻辑交互结构选择初始布局	减少从不合适起点开始搜索带来的额外交换。
束搜索	从单步选择扩展到多候选路线保留	降低局部最优对长距离交互线路的影响。
学习评分	用动作掩码、模型评分和局部修复增强搜索	在可解释搜索框架内提高候选动作排序质量。

4.1 路由闭环

路由闭环解决的是“能否正确路由”的问题。给定输入线路 C 和硬件拓扑 G ，系统先提取门依赖关系，得到当前可以尝试执行的前沿门集合；再维护映射

$$\pi_t : Q \rightarrow V,$$

表示第 t 步时每个逻辑量子比特所在的物理位置。若前沿中的双比特门 $g = (q_i, q_j)$ 满足

$$(\pi_t(q_i), \pi_t(q_j)) \in E,$$

系统直接执行该门并推进依赖关系。若条件不满足，系统必须在硬件边上插入 SWAP，更新映射后继续检查前沿。

这个闭环把路由过程写成了一个状态转移问题。状态至少包含当前映射、已执行门集合、待执行门集合和已插入的 SWAP 序列。每次插入 SWAP，相当于沿某条物理边

交换两个逻辑量子比特的位置；每次执行可行双比特门，相当于把线路依赖图向前推进一步。只要所有原始门都被执行，并且每一步双比特操作都满足硬件边约束，输出线路就是拓扑合法的。

表 4: 组件职责

模块	输入	输出	作用
依赖关系	门序列	前沿门、前驱关系	判断哪些门可以在当前状态推进。
拓扑预处理	硬件耦合图	距离矩阵、边集合	为动作评分和合法性检查提供图特征。
初始映射	逻辑交互图、物理拓扑	候选初始布局	降低后续 SWAP 搜索压力。
动作筛选	当前映射、前沿门	候选 SWAP 集合	删除弱相关动作，控制分支数量。
模型评分	状态特征、候选动作	动作偏好分数	为搜索提供学习得到的局部判断。
束搜索	候选状态、累计得分	多条候选路线	保留多条高分路线，提升搜索质量。
局部修复	接近完成的路线	修复后的尾部轨迹	处理少量剩余门停滞的情况。
轨迹复放	执行轨迹、硬件拓扑	合法性检查结果	确认最终线路满足拓扑约束。

4.2 前沿门推进

表 4 按功能列出路由闭环和后续增强模块。这里不按程序文件组织，而按算法职责理解：依赖图负责判断前沿，拓扑预处理负责提供距离，初始映射负责选择起点，动作筛选和评分负责控制搜索分支，轨迹复放负责验证最终结果。

前沿门是路由过程的节拍器。对于一个门，如果它所有前驱都已经执行，它就进入前沿。单比特门不受硬件耦合边限制，通常可以直接推进；双比特门还需要检查当前映射下的两个物理位置是否相邻。这个判定把课程中的门依赖关系和硬件拓扑约束连接起来。

系统在每个状态都会尽量执行已经可行的前沿门。这样做有两个好处。第一，它避免对已经合法的门插入多余 SWAP，使搜索只关注真正受拓扑限制的部分。第二，它减少后续状态中的未完成门数量，让动作选择围绕当前瓶颈展开。若前沿双比特门尚不可行，系统才进入候选 SWAP 生成过程。

物理距离矩阵在前沿判断中也起到辅助作用。对一个尚不可行的前沿门 $g = (q_i, q_j)$,

距离

$$d_G(\pi(q_i), \pi(q_j))$$

表示它离可直接执行还差多少拓扑移动。这个距离不是最终目标函数，但它给后续动作筛选和评分提供了直接、稳定的启发式信号。

4.3 初始映射

基础闭环可以从简单初始映射开始运行，但起点质量会显著影响后续 SWAP 数。若逻辑线路中的高频交互量子比特一开始被放到硬件拓扑的远端，搜索过程就需要先花大量 SWAP 把它们拉近。对于 QFT、QAOA、VQE 和随机线路，这种起点差异尤其明显，因为它们的双比特交互结构并不相同。

NPQR 使用逻辑交互图来改进初始映射。逻辑交互图把每个逻辑量子比特看作顶点，把双比特门看作带权边；边权可以由两个量子比特之间出现的双比特门次数或交互强度给出。设逻辑交互权重为 w_{ij} ，一个直观的初始映射评分可以写成

$$S(\pi) = \sum_{i < j} w_{ij} d_G(\pi(q_i), \pi(q_j)).$$

评分越小，说明高频交互对在硬件上越接近。实际选择初始映射时，系统会生成多个候选布局，并按交互距离、最大距离和远距离交互数量进行排序，再把较好的候选交给后续搜索。

这种映射改进体现了从“能路由”到“少交换”的第一步变化。路由闭环只保证每一步合法；初始映射进一步利用线路的整体交互结构，减少搜索一开始就背负的距离代价。后续实验中的阶段耗时也显示，结构化线路的运行时间常集中在初始映射候选生成上，这说明起点选择已经成为当前系统质量和成本的重要部分。

4.4 搜索策略

在初始映射之后，路由器仍要面对局部选择问题。单步贪心只保留当前分数最高的动作，容易在长距离交互或密集前沿中走入局部最优。NPQR 因此使用有界搜索保留多条候选路线，用束宽控制质量和耗时之间的平衡。这里的“有界”很关键：搜索不是枚举所有 SWAP 序列，而是在每一步只展开经过筛选的候选动作，并保留累计得分靠前的状态。

图 4 给出搜索过程。每个状态先推进可执行前沿门；若仍存在不可执行的双比特门，系统从硬件边中生成候选 SWAP，并根据当前前沿门、最近后继门和物理距离筛掉弱相关动作。保留下来的动作会产生新的映射状态。束搜索把这些状态按累计得分排序，只保留前 B 条路线进入下一次展开。这样既能避免单步选择过早定局，也能把分支数量控制在可运行范围内。

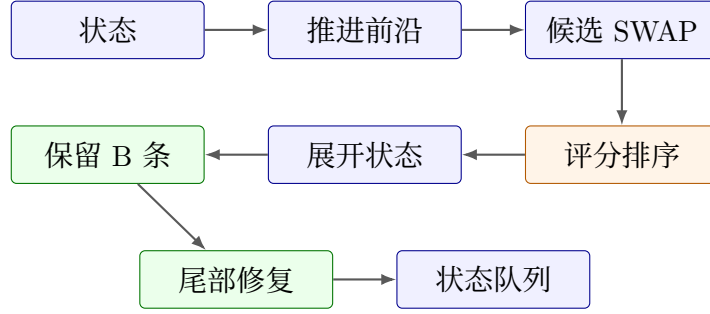


图 4: 搜索过程

设状态 s_t 的累计代价为 $J(s_t)$ ，从该状态选择动作 a 后得到 s_{t+1} 。搜索过程可以写成

$$J(s_{t+1}) = J(s_t) + c(a, s_t),$$

其中 $c(a, s_t)$ 是对当前动作的局部代价估计。它既考虑新增 SWAP 的直接成本，也考虑动作对前沿门距离的改善。对一个候选动作 $a = (u, v)$ ，若交换 u 和 v 上的逻辑量子比特后能显著缩短前沿双比特门的距离，这个动作会得到较低的代价；若它只移动与当前依赖无关的量子比特，代价会升高。

这种搜索改进保留了 SABRE 启发式的核心直觉，即围绕前沿门和后继门估计 SWAP 价值，但把决策从“只选一个动作”扩展为“保留一组候选路线”。在 QFT 和随机线路这类长程交互较多的样例中，早期的一个 SWAP 选择会影响后续多轮门的可执行性，束搜索能把这种长期影响保留到后续状态中再比较。

4.5 动作评分

动作评分部分负责给候选 SWAP 动作提供学习得到的偏好。模型输入来自当前映射、前沿门、物理距离和候选动作特征，输出作为动作排序的一部分。它不单独决定最终线路，而是和距离启发式、动作掩码、束搜索以及局部修复共同工作。

动作掩码先定义模型可以评价的动作集合。由于 SWAP 只能发生在硬件边上，所有非边动作都会被直接排除；同时，若某条边交换的是两个与当前前沿及近邻后继都无关的量子比特，它通常不会进入优先候选。掩码的作用不是替模型做最终判断，而是把模型推理限制在物理合法且与当前瓶颈相关的动作上。

在候选集合上，NPQR 将模型分数和启发式分数合成一个动作分数：

$$F(a | s) = \lambda f_\theta(a, s) - \alpha \Delta d_{\text{front}}(a, s) - \beta \Delta d_{\text{future}}(a, s) - \gamma r(a, s).$$

其中 $f_\theta(a, s)$ 表示模型对动作的偏好， Δd_{front} 表示前沿门距离的改善， Δd_{future} 表示近邻后继门距离的改善， $r(a, s)$ 表示反复交换、远离关键交互等惩罚项。这个公式的目的不是把路由问题黑箱化，而是把可解释的距离变化和学习得到的动作偏好放在同一个排序规则中。

学习评分带来的主要变化是候选动作的排序方式。纯距离启发式更容易偏向眼前最

近的前沿门；模型分数则可以利用训练过程中见过的局部结构，例如某些 SWAP 虽然短期内没有立刻执行一个门，却能让后续一组门同时接近可执行状态。最终路线仍由束搜索、拓扑合法性验证和轨迹复放决定，因此模型只提供偏好，不改变路由问题的约束。

4.6 局部修复

束搜索在大多数样例中能找到完整路线，但在少数尾部状态中会出现“只剩几个门却反复移动”的情况。NPQR 在这种状态上使用局部修复：当未完成门数量较少、前沿门距离不大且主搜索长时间没有推进时，系统临时切换到受深度限制的后缀搜索。修复搜索只关注剩余门及其相关量子比特，允许在较小深度内尝试几条直接把前沿门拉近的 SWAP 序列。

局部修复的定位是补足主搜索的尾部能力，而不是替代主搜索。它的搜索深度受到固定上限限制，只有在路线已经接近完成时才触发。这样可以避免为少数困难尾部显著扩大整体搜索空间，同时提高输出线路的完成率和轨迹连续性。

4.7 复放验证

输出线路必须经过轨迹复放。复放从初始映射开始，按顺序重走每一个原始门和每一个 SWAP，检查双比特操作是否位于硬件边上，并确认所有原始门都被执行。复放通过后，SWAP 数、深度和最终映射才作为结果进入实验表格。这个步骤把算法搜索结果转换成可复放的物理执行轨迹。

复放还承担语义一致性验证。路由过程不会删除原始量子门，也不会改变原始门之间的依赖顺序；新加入的操作只是在硬件边上交换逻辑量子比特的位置。复放按同一套映射更新规则重走轨迹，可以验证每次 SWAP 后逻辑量子比特的位置是否与后续门执行一致。因此，实验表中的 SWAP 数和深度不是单纯的搜索日志统计，而是来自已经复放通过的路由结果。

4.8 算法过程

算法 1: NPQR 路由

输入: 量子线路 C , 硬件图 G , 束宽 B , 修复深度 R

输出: 路由后线路、SWAP 数、执行轨迹或状态标记

1. 解析 C , 构建线路依赖关系和前沿门集合;
2. 计算 G 上的距离矩阵, 生成逻辑交互图;
3. 生成并排序初始映射候选, 形成初始状态集合;
4. **while** 存在待处理状态 **do**
 - 4.1 执行当前所有可直接执行的前沿门;
 - 4.2 若状态完成, 复放执行轨迹并返回成功结果;
 - 4.3 生成合法 SWAP 候选并筛选动作;
 - 4.4 结合模型偏好和距离启发式为动作评分;
 - 4.5 展开候选状态, 保留累计评分较高的 B 条路线;
 - 4.6 若路线接近完成但停滞, 尝试深度不超过 R 的局部修复;
5. **return** 状态标记。

图 5: 算法过程

5 实验系统设计

算法章节说明了 NPQR 如何产生一条合法路由轨迹。本章讨论系统形态: 同一套编译核心如何被网页、接口、协议工具和命令行复用, 并把量子线路、硬件拓扑、路由过程和结果指标展示出来。系统设计的目标不是只给出一个离线实验脚本, 而是让报告中的方法能够被打开、调用、观察和复现。

图 6 给出系统结构。网页演示、REST 服务、MCP 服务和命令行都进入统一编译请求; 请求经过 NPQR 路由运行时后, 得到路由线路、SWAP 数、线路深度、初始映射、最终映射和轨迹事件; SABRE basic 在同一接口中作为固定基线返回, 用于直接比较。

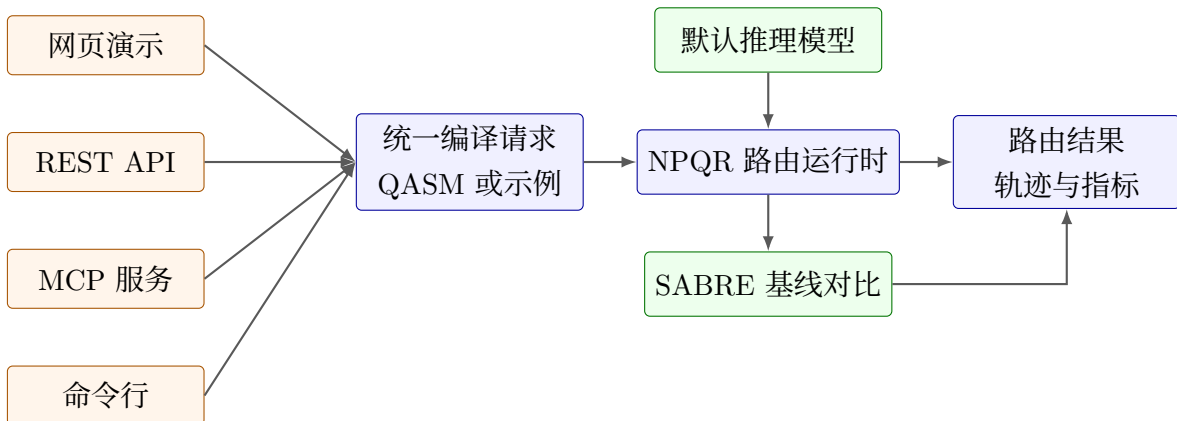


图 6: 系统结构

5.1 演示界面

网页演示面向课程展示和实验复现。页面保留三个核心工作区：左侧输入或选择 OpenQASM 线路，中间展示硬件拓扑与路由回放，右侧展示指标、基线对比和编译后线路。读者可以选择内置样例，也可以粘贴自己的小规模线路；系统会根据线路规模选择合适拓扑，并把编译结果返回到同一个界面。

这个界面的重点是“看得见的路由”。当编译完成后，页面先显示初始映射，再按轨迹事件逐步高亮 SWAP、双比特门和物理边。对于量子线路路由问题，单独给出一个 SWAP 数并不足以说明算法行为；轨迹回放能显示逻辑量子比特如何沿硬件边移动，也能直观看到哪些门在映射调整后变得可执行。

5.2 调用方式

除网页外，系统还提供三类调用方式。REST 服务用于网页和普通后端程序调用，适合查询示例、校验线路、同步或异步发起编译任务。MCP 服务面向工具客户端，把状态查询、样例列表、NPQR 编译、SABRE 编译和结果摘要整理成可调用工具。命令行用于本地状态确认、示例编译和批量结果生成。

表 5 列出各调用方式共同返回的主要信息。无论从网页、服务接口还是命令行进入，核心字段保持一致：路由后线路、SWAP 数、深度、耗时、映射、轨迹、SABRE 对比和完成状态。统一字段可以减少演示和实验之间的口径差异，并使同一结果能够进入报告表格。

表 5: 接口输出

返回信息	用途
路由后 QASM	作为后续量子工具链可继续处理的编译结果。
初始/最终映射	显示逻辑量子比特如何放置到物理量子比特。
SWAP 数	衡量硬件拓扑限制带来的额外交换门数量。
线路深度	衡量路由后执行层数，间接反映噪声暴露时间。
运行时间	衡量在线调用和课堂演示中的响应成本。
route trace	记录每一步门执行和 SWAP 插入，支持回放验证。
SABRE 指标	提供固定基线对照，帮助解释 NPQR 的结果质量。
完成状态	标记编译是否完成并通过基本约束验证。

5.3 拓扑与样例

系统支持两类拓扑。20 比特以内的课程样例默认使用 Tokyo 20Q 拓扑，用来模拟真实量子芯片的非完全耦合结构；30 比特和 50 比特扩展样例使用规则网格拓扑，用于观察较大线路在受限平面结构上的路由成本。拓扑不是单纯的显示元素，而是贯穿合法性验证、距离矩阵、动作筛选和轨迹复放的约束来源。

网页会把拓扑选择和线路规模绑定。小规模线路可以直接在 Tokyo 拓扑上观察复杂耦合结构，较大线路则进入对应网格。这样做避免了线路量子比特数超过物理量子比特数的无效输入，也让实验章节中的 10/20Q 主结果和 30/50Q 扩展结果拥有清晰的硬件背景。

5.4 轨迹回放

轨迹回放是系统最能体现量子路由过程的功能。每个事件包含当前步骤、操作类型、涉及的逻辑量子比特、对应物理位置和映射变化。对于原始双比特门，回放验证两个物理位置是否相邻；对于 SWAP，回放验证交换边是否属于硬件拓扑。通过这些事件，读者可以从界面上追踪“为什么这里需要交换”和“交换后哪个门可以执行”。

这个设计也服务于实验复现。报告中的 SWAP 数和线路深度来自同一条轨迹，而不是与界面无关的单独统计。当前端显示 NPQR 与 SABRE basic 的比较时，二者使用相同的线路输入、拓扑和指标字段，因此读者可以把网页结果、接口返回和报告表格对应起来。

5.5 功能结构

图 7 按功能而非文件列出系统结构。算法层负责线路解析、依赖关系、初始映射、动作评分、束搜索和轨迹回放；服务层负责把结果提供给网页、REST、MCP 和命令行；证据层保存示例、测试和实验摘要。这样的组织方式让系统既能作为量子信息基础大作业展示，也能作为后续继续实验的基础。

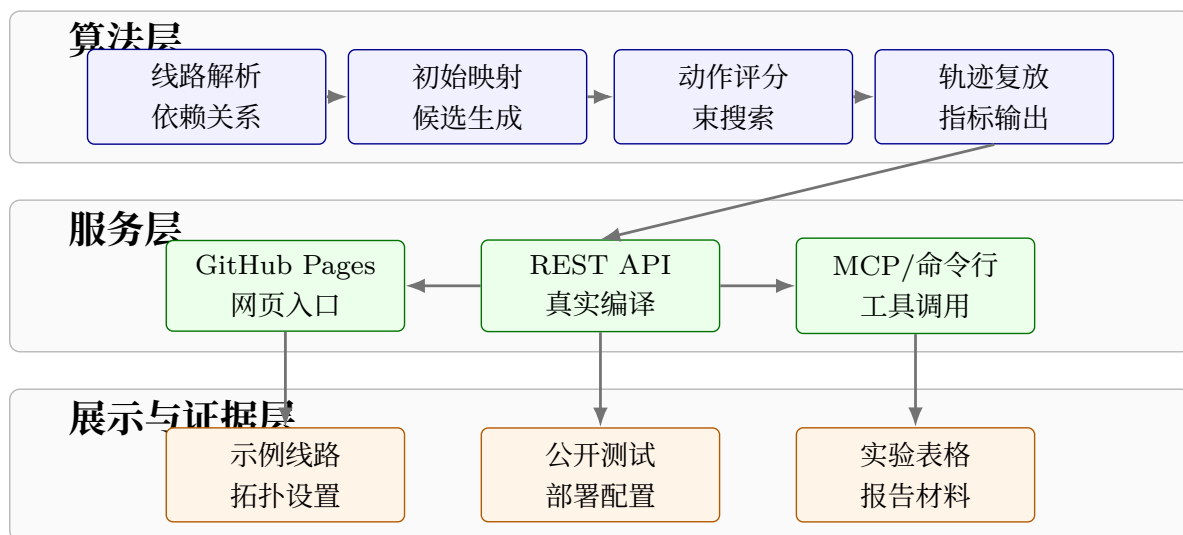


图 7: 功能结构

6 项目部署与交付

本章说明项目如何从本地算法实现变成可打开、可运行、可复现的量子信息基础大作业成果。交付形态包括公开仓库、在线前端、服务器接口、协议工具、命令行、测试验证和报告 PDF。这些材料共同支撑网页体验、接口调用和本地复现实验。

6.1 项目组织

公开仓库是项目总览。仓库首页说明项目目标、安装方式、在线演示、服务接口、命令行用法、样例线路、拓扑选择和主要实验结果。报告 PDF、网页演示、示例线路、测试脚本和说明文档都放在同一仓库中，避免出现“报告写了一套、代码在另一套环境中”的问题。

项目组织遵循量子信息基础大作业的交付逻辑：仓库说明项目目标和运行方式，在线前端呈现交互效果，命令行和服务接口支持复现实验，公开测试维持说明、接口和示例的一致性。这个结构把算法、系统和实验联系在一起。

图 8 展示公开仓库入口。仓库首页把 README、许可证、目录结构、在线前端和提交历史放在同一视图中，读者进入仓库后可以先阅读快速入口，再根据需要打开网页、启动服务或运行本地测试。

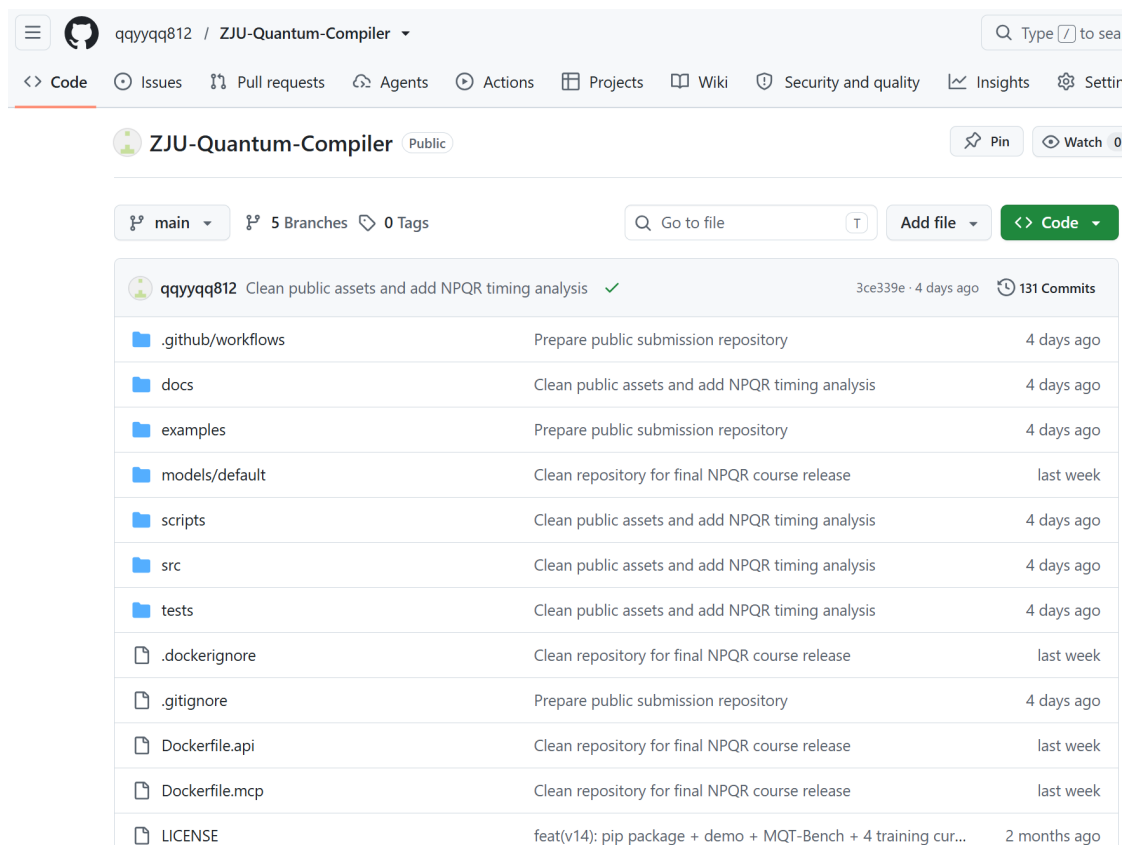


图 8: 仓库入口

图 9 展示项目周期。整体工作先完成 SABRE 基线与最小路由闭环，再围绕 NPQR 增加初始映射、模型评分、束搜索、修复和复放；随后把算法封装为网页、REST、MCP 和命令行；最后整理公开仓库、在线前端、服务器服务、测试验证和报告材料。

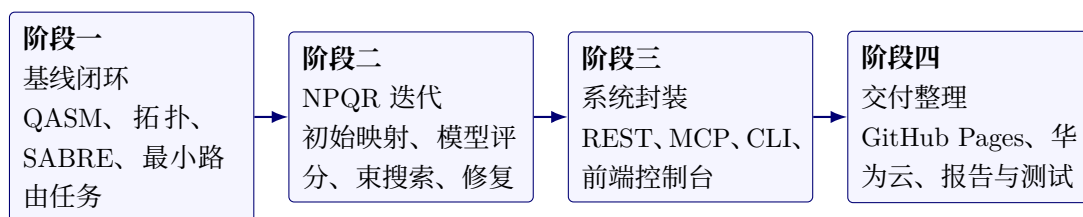


图 9：项目周期

6.2 在线前端

前端采用静态网页发布，适合课程展示和跨设备访问。静态页面不要求访问者先配置本地环境，打开后即可查看示例、拓扑、指标面板和轨迹回放。当服务器接口可达时，页面会把编译请求发送到后端；当读者只想了解功能时，页面中的样例和说明也能独立展示项目结构。

GitHub Pages 适合承载这类公开展示页面：它与公开仓库天然绑定，更新流程清晰，访问链接固定。前端不保存实验结论本身，而是作为编译结果的展示层；真正的路由、基线比较和轨迹生成仍由共享编译核心完成。

6.3 服务器部署

服务器部署负责处理真实编译请求。后端服务接收线路、后端选择和拓扑名称，调用同一编译核心，并返回路由后线路、指标和轨迹。服务端提供健康状态、样例列表、拓扑查询、线路校验、同步编译、异步任务和实验摘要等能力。异步任务用于耗时稍长的线路，避免浏览器在等待过程中失去响应。

云端部署使用容器化方式固定运行环境。这样做有两个优点：一是依赖库、模型文件和服务启动方式可以被明确记录；二是本地运行和服务器运行使用同一套调用方式，减少“本地能跑、线上不一致”的问题。报告中的在线前端与服务器接口保持分工：前端负责交互，服务器负责计算。

6.4 工具接口

除普通网页和 REST 调用外，项目还提供 MCP 与命令行两类接口。MCP 面向支持工具调用的客户端，适合把量子编译能力接入自动化 workflow；命令行面向本地复现，用于确认安装状态、编译内置样例和生成对照矩阵。

这两类接口的价值在于扩展使用场景。网页适合展示，REST 适合应用集成，MCP 适合工具编排，命令行适合复现实验。四种调用方式共享同一结果字段，因此不会形成互相矛盾的结果口径。

6.5 测试与复现

公开交付包含一组基础测试。测试覆盖服务契约、网页静态契约、README 内容、部署配置和材料完整性。这里的测试目标不是证明算法在所有可能线路上最优，而是保证说明、接口和示例能够互相对应。

表 6 汇总主要交付项。正文只解释交付结构，具体命令放在附录中，作为复现实验的命令说明。

表 6: 交付清单

交付项	作用
公开仓库	汇总代码、报告、样例、测试和说明文档。
在线前端	展示线路输入、拓扑选择、轨迹回放和指标对比。
服务器接口	处理真实编译请求，返回路由线路、指标和轨迹。
MCP 服务	为工具客户端提供状态查询、样例读取和编译调用能力。
命令行接口	支持本地安装确认、示例编译和对照矩阵生成。
测试验证	覆盖公开接口、网页契约、README 和材料完整性。
报告 PDF	汇总理论基础、SABRE 复现、NPQR 方法、系统部署和实验结果。

7 AI 辅助开发

本项目把 AI 工具作为工程协作环境的一部分，而不是把它作为算法结论的来源。量子线路路由同时涉及量子信息、图搜索、学习模型、前后端接口和部署流程，单靠一次性编写代码很难把这些部分稳定连接起来。AI 的作用主要体现在信息整理、方案比较、代码阅读、界面修订和文档组织上；项目目标、算法取舍、实验口径和最终文字由作者根据课程要求和实际运行结果确定。

7.1 资料整理

项目早期需要把课程知识、经典路由算法和工程实现放在同一问题框架中。AI 辅助完成的第一类工作是资料整理：把量子态、酉门、双比特门、SWAP、硬件耦合图和线路深度这些课程概念对应到路由编译任务；把 SABRE 的前沿门、候选交换和距离启发式整理成可复现的基线描述；再把 NPQR 的初始映射、束搜索、动作评分和轨迹复放到“学习评分搜索”这一叙事下。

这种整理不是替代阅读论文，而是帮助形成报告结构。SABRE、图神经网络辅助路由和强化学习路由等相关方法提供了参考坐标：传统启发式强调速度和稳定性，学习方法强调从局部结构中学习动作偏好，搜索框架负责保证拓扑约束和状态推进。本项目最终选择的写法是先复现 SABRE，再说明自研方法如何在同一实验口径下逐步改进。

7.2 网页开发

网页演示需要让读者直观看到量子路由的过程，而不是只看到一个数字表格。AI 在前端部分主要用于交互方案和界面细节的讨论：输入区如何容纳 OpenQASM，拓扑图和指标面板如何并列显示，轨迹回放如何同步显示 SWAP、物理边和逻辑位交换，错误信息如何写得清楚但不干扰主流程。

前端技术栈保持简单：静态页面负责展示和交互，浏览器脚本负责请求服务接口、渲染拓扑、回放轨迹和更新指标。这样的选择适合量子信息基础大作业，因为它不要求额外的前端构建流程，并且可以直接通过 GitHub Pages 发布。AI 辅助的价值主要体现在快速比较布局方案、核对按钮状态、梳理响应字段和改写界面文案；最终页面是否可用，仍由实际浏览器打开、接口调用和样例回放决定。

7.3 服务开发

后端服务承担真实编译任务。AI 在这一部分更多地用于代码阅读和接口一致性梳理：梳理 REST、MCP 和命令行三种调用方式需要共享哪些字段，整理状态查询、示例列表、线路校验、同步编译、异步任务和拓扑查询之间的返回格式是否一致，帮助发现前端字段和后端响应之间的不匹配。

服务端技术栈包括 Python、FastAPI、Uvicorn、Qiskit、PyTorch、NetworkX 和 Rustworkx。其中 Qiskit 用于量子线路处理和 SABRE 基线，图工具用于硬件拓扑和距离计算，PyTorch 用于模型评分模块，FastAPI 和 Uvicorn 用于把编译能力暴露为服务。AI 辅助在这里更像一名工程搭档：它可以根据接口约定整理验证项，但每一次接口修改都需要通过实际请求、测试用例和网页调用来确认。

7.4 部署测试

部署阶段容易出现两类问题：一类是本地可运行但服务器环境缺少依赖，另一类是 README、网页、接口和测试之间的说明不一致。AI 在这一阶段用于整理验证流程，例如服务健康状态、样例编译、静态页面契约、公开测试和材料完整性。容器化服务、GitHub Pages 和公开仓库共同构成了本项目的交付链路。

这一过程也帮助项目把“能运行”扩展为“能交付”。本地命令行适合快速复现实验，REST 服务适合网页和脚本调用，MCP 适合工具客户端集成，GitHub Pages 适合课程展示。AI 可以辅助列出这些调用方式之间需要保持一致的结果字段，但真正的通过标准是服务能够启动、网页能够调用、测试能够运行、报告中的实验数据能够对应到实际输出。

7.5 报告整理

报告写作阶段，AI 主要用于结构重组和文字修订。早期材料更像项目说明，包含较多接口、路径和过程记录；正式报告需要改成量子信息基础大作业文风：先讲量子信息基础和硬件约束，再讲 SABRE 复现、自研方法、系统部署和实验对照。AI 在这里帮助发现章节之间的断裂，例如摘要是否回应课程要求，图表标题是否过长，实验结论是否对应表格，系统与部署是否混在同一节中。

最终写法遵循两个原则。第一，正文解释问题和方法，具体复现命令放到附录；第二，图表标题保持简短，详细解释放在段落中。这样可以减少工程文件列表对正文节奏的干扰，让报告更接近课程论文，而不是开发日志。

7.6 过程控制

AI 协作最关键的部分是控制权。作者负责确定量子信息基础大作业的目标、选定对照基线、决定哪些实验进入正文、运行测试和编译 PDF，并对最终结论负责。AI 给出的代码方案、文字方案和实验组织方案只有在符合实际代码、样例输出和课程要求时才会被采用。

因此，本项目中的 AI 使用方式可以概括为：用 AI 提高阅读、开发和整理效率，用代码运行结果决定结论。对于量子线路路由这样涉及算法与工程闭环的问题，这种协作方式能够减少重复劳动，也能帮助报告保持清晰结构；但项目可信度仍来自可运行系统、可复现实验和明确的课程知识对应关系。

8 实验结果与数据分析

实验部分围绕三个问题展开。第一，NPQR 在代表性线路上是否能生成通过复放的路由结果。第二，在同一线路和同一拓扑下，NPQR 相比 SABRE basic 能减少多少 SWAP。第三，初始映射、束搜索、模型评分和局部修复这些设计在运行成本和结果质量上分别起到什么作用。

8.1 实验设置

主实验采用固定对照口径。20 比特以内线路使用 Tokyo 20Q 拓扑，30/50 比特扩展线路使用规则网格拓扑；NPQR 与 SABRE basic 使用相同输入线路、相同拓扑和相同指标字段。SABRE basic 作为固定基线，只用于质量比较；NPQR 路线由自研流程生成，并经过轨迹复放后进入表格。

表 7: 指标

指标	含义	解释方式
完成状态	是否生成满足拓扑约束的完整路由结果	路由实验的先决指标。
轨迹复放	按路由轨迹重新验证每一步门和 SWAP	用于确认结果来自完整物理轨迹。
SWAP 数	插入的交换门数量	越低通常表示额外路由开销越小。
线路深度	路由后门序列的执行层数	越低通常表示潜在执行时间和噪声暴露越少。
运行时间	编译请求耗时	反映在线接口和演示场景的响应成本。
基线差异	NPQR 与 SABRE basic 的指标差值	用于观察质量优势和路由开销变化。

实验指标见表 7。完成状态是先决指标：只有所有原始门被执行、每个双比特门都

满足硬件边约束、轨迹可以从初始映射复放到最终映射时，结果才记为完成。在完成前提下，SWAP 数衡量额外交换开销，线路深度衡量可并行执行层数，运行时间衡量在线演示和批量复现实验的计算成本。

8.2 基础实验

10/20 比特实验是本报告的主要质量证据。样例覆盖 GHZ、QFT、QAOA、VQE、随机线路和结构化 20 比特线路，分别对应链式纠缠、长程交互、局部哈密顿量结构、变分线路和较不规则的双比特门分布。

表 8: 主实验

线路	比特数	NPQR SWAP	SABRE basic SWAP	差值
GHZ10	10	2	6	-4
QFT10	10	6	13	-7
QAOA10	10	0	15	-15
VQE10	10	1	4	-3
Random10	10	17	19	-2
Brickwork20	20	15	31	-16
CliqueBlocks20	20	19	38	-19
DeepRandom20	20	105	135	-30
RingEntangler20	20	16	41	-25
SparseRandom20	20	37	57	-20

表 8 显示，NPQR 在 10 个代表样例上全部完成路由，SWAP 数均低于 SABRE basic。10 个样例合计，NPQR 插入 218 个 SWAP，SABRE basic 插入 359 个，减少 141 个，降幅为 39.3%。其中 10 比特样例从 57 个降至 26 个，20 比特样例从 302 个降至 192 个。

图 10 选取不超过 70 个 SWAP 的代表行画成柱状图，避免 DeepRandom20 这类大值压缩其他样例的视觉差异。结构化线路的收益最明显：QAOA10 从 15 个 SWAP 降至 0 个，CliqueBlocks20 从 38 个降至 19 个，RingEntangler20 从 41 个降至 16 个。随机线路的改善幅度相对小一些，但仍保持同向：Random10 减少 2 个 SWAP，SparseRandom20 减少 20 个 SWAP。这说明 NPQR 的收益并不只来自某一个特定样例，而是同时出现在结构化线路和随机线路中。

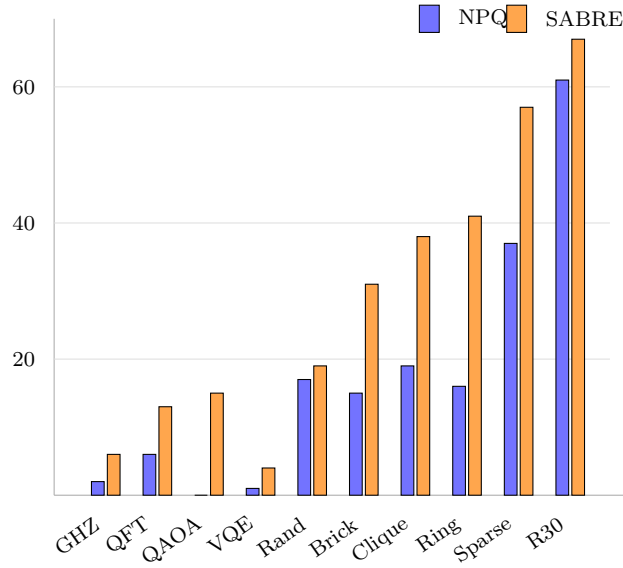


图 10: SWAP 对比

8.3 拓展实验

扩展实验选择 30/50 比特网格样例，用来观察同一套路由流程在更大拓扑上的表现。这里不把扩展样例作为全规模能力结论，而是作为补充实验：线路规模增大后，初始映射和搜索仍需在受限耦合图上给出可复放路线。

表 9: 扩展实验

线路	比特数	NPQR SWAP	SABRE basic SWAP	差值
LineGHZ30	30	23	43	-20
Random30-d4	30	61	67	-6
LineGHZ50	50	50	79	-29
RingSparse50	50	98	113	-15

表 9 中 4 个扩展样例全部完成，且 SWAP 数均低于 SABRE basic。LineGHZ30 从 43 个 SWAP 降至 23 个，LineGHZ50 从 79 个降至 50 个；这类链式纠缠线路受益于较好的初始布局。Random30-d4 和 RingSparse50 的改善幅度较小，分别减少 6 个和 15 个 SWAP，说明在较不规则交互下，NPQR 仍能形成优势，但搜索压力更高。

8.4 消融实验

消融实验采用 10 比特子集，固定 Tokyo 20Q 拓扑和相同最大步数，只改变路由组件。该组实验用于观察模块作用，不替代主实验结论。完整配置包含结构化初始映射、多候选束搜索和模型动作评分；单步选择把束宽和分支数都降为 1；简单映射减少初始映射候选和局部布局优化。

表 10: 消融实验

配置	完成数	完成样例	SWAP 合计	观察
完整配置	4/4		23	GHZ10、QFT10、QAOA10、VQE10 全部完成。
单步选择	1/4		0	只完成 QAOA10，长程交互样例容易停滞。
简单映射	2/4		22	QFT10 和 QAOA10 完成，但 QAOA10 需要额外 SWAP。

表 10 显示，完整配置在 4 个样例上全部完成。去掉多候选保留后，只有 QAOA10 完成；GHZ10、QFT10 和 VQE10 都在有限步数内停住。弱化初始映射后，完成数提高到 2 个，但 GHZ10 和 VQE10 仍未完成，QAOA10 的 SWAP 数也从 0 增加到 3。这个结果支持方法章节中的判断：束搜索负责避免过早陷入局部路线，结构化映射负责降低搜索起点的距离代价。

模型评分另做了动作价值信号分析。对 15 组正负路线状态，模型在 12 组中给可取路线更高分，正向比例为 80%，平均分差为 0.00213。这个数值不直接等同于 SWAP 降幅，但说明模型评分具有可用于排序的局部信号；最终路线质量仍由动作掩码、束搜索、修复和复放共同决定。

8.5 运行过程

为了解释运行成本，项目对 NPQR 主流程做了分阶段计时。计时覆盖预处理、依赖关系、拓扑距离矩阵、逻辑交互图、初始映射候选、主搜索、动作生成、模型推理、束搜索剪枝、后处理、局部修复和轨迹复放。表 11 选取三个代表样例：QAOA10、QFT10 和 Brickwork20。

表 11: 阶段耗时

样例	规模	总耗时	最耗时阶段	搜索子阶段观察
QAOA10	10Q	623.9ms $\pm$ 61.2ms	初始映射候选生成 92.6%	结构化线路主要由初始映射吸收路由成本。
QFT10	10Q	16.23s $\pm$ 1.14s	主搜索阶段 90.2%	模型推理约 7.75s，束搜索扩展与剪枝约 5.65s。
Brickwork20	20Q	13.35s $\pm$ 0.86s	初始映射候选生成 99.8%	20Q 候选映射成为主要成本。

图 11 展示三类样例的瓶颈差异。QAOA10 和 Brickwork20 的耗时集中在初始映射候选生成，说明结构化交互线路的主要成本在起点选择；QFT10 的主搜索占 90.2%，其中模型推理和束搜索扩展是主要子项。这个结果与复杂度分析一致：当线路包含更多长程依赖时，候选状态数量和动作评分次数会成为主要增长项。

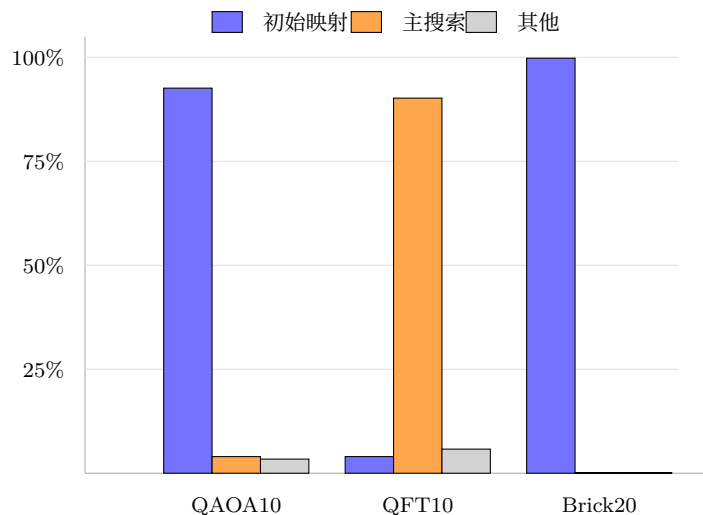


图 11: 耗时分布

实验总体说明，NPQR 的质量收益来自多个组件的组合，而不是单一技巧。初始映射减少搜索起点的拓扑距离，束搜索保留多条候选路线，模型评分提供局部动作偏好，局部修复处理尾部停滞，轨迹复放保证结果可验证。运行时间仍有优化空间，尤其是候选映射生成、模型推理和束搜索扩展；这些开销也为后续批处理推理、候选剪枝和分层路由提供了明确方向。

9 实验讨论

本节把 NPQR 的工程实现回扣到算法分析和量子信息基础知识。报告的重点不是堆叠公式，而是说明为什么某些阶段耗时高、为什么 SWAP 数和线路深度有物理意义，以及本项目与课程内容之间的对应关系。

9.1 复杂度

令 $n = |V|$ 表示物理量子比特数， m 表示线路门数， $e = |E|$ 表示硬件边数， K 表示初始映射候选数， B 表示束宽， D 表示主搜索最大展开深度， A 表示每个状态保留的候选动作数， R 表示后缀修复深度。NPQR 的主要成本来自候选映射、动作生成与掩码、模型推理、束搜索扩展和局部修复。

表 12: 复杂度

阶段	时间复杂度估计	说明
拓扑距离矩阵	$O(n^3)$ 或 $O(n(e + n \log n))$	取决于 Floyd 或多源最短路；固定拓扑下可复用。
线路依赖图构建	$O(m)$	扫描门序列，维护前沿门和执行依赖。
逻辑交互图构建	$O(m)$	统计双比特门交互强度，服务初始映射。
初始映射候选	$O(K \cdot n^2)$ 量级	与候选数量和映射评分方式有关。
动作生成与掩码	$O(B \cdot e)$	每个保留状态在硬件边上筛选候选 SWAP。
模型推理	$O(B \cdot A \cdot C_\theta)$	C_θ 为单次模型评分成本。
束搜索扩展	$O(D \cdot B \cdot A \cdot C_s)$	C_s 包括状态复制、执行门和评分更新。
后缀修复	$O(A^R)$ 的受限搜索	只在接近完成时启动，避免全局穷举。
轨迹复放	$O(m + N_{\text{swap}})$	线性验证输出轨迹是否合法。

表 12 中的估计解释了实验计时结果。拓扑距离矩阵和线路依赖图构建属于固定预处理，规模可控；候选映射和主搜索会随线路结构快速变重。束宽 B 和动作数 A 增大时，搜索质量可能提高，但状态数量和模型评分次数也随之增加。后缀修复在最坏情况下呈指数增长，因此只用于接近完成的局部尾部。

9.2 方法对比

表 13: 方法对比

方法	核心思想	优点	本项目中的角色
SABRE	基于前沿门和后续门距离选择 SWAP	稳定、快速、工程常用	固定质量基线，不参与 NPQR 路线生成。
QRoute	图神经网络辅助蒙特卡洛树搜索	结合学习模型与树搜索	相关方法参考。
AlphaRouter	强化学习结合搜索策略	强调策略学习和多步规划	相关方法参考。
NPQR	模型评分、初始映射、束搜索、剪枝和修复组合	可解释、可接口调用、可复放验证	项目自研默认路由流程。

SABRE 的核心是基于前沿门和后续门距离的启发式 SWAP 选择，优点是速度快、实现成熟。NPQR 的区别在于把动作评分的一部分交给模型学习，同时保留显式搜索、剪枝、修复和复放验证。两者的比较说明一个工程事实：神经网络评分需要与拓扑约束、动作掩码和搜索过程结合，才能产生可复放的合法线路。

9.3 结果解释

基于实验和复杂度分析，NPQR 在 10/20 比特代表样例和选定 30/50 比特网格样例上取得了低于 SABRE basic 的 SWAP 数。它的运行时间主要消耗在候选映射、模型推理和束搜索上。这个结果与项目定位一致：系统优先展示可复放路线、可解释搜索过程和多种接口交互能力，并把编译速度优化留给后续工程迭代。

10 总结与体会

本项目围绕量子线路在受限硬件拓扑上的路由编译展开，把量子信息基础中的线路模型、双比特门、SWAP、硬件耦合图和线路深度落实到一个可运行系统中。报告先说明真实芯片为什么需要路由，再复现 SABRE basic 作为传统启发式基线，随后给出 NPQR 的完整流程：初始映射选择起点，前沿推进维护线路依赖，动作评分对候选 SWAP 排序，束搜索保留多条路线，局部修复处理尾部停滞，复放验证保证输出线路满足拓扑约束。

量子信息基础大作业要求的理论、算法、实验和工程交付在项目中形成了闭环。理论部分覆盖量子态、量子门、双比特门、SWAP、拓扑映射和路由代价；算法部分完成 SABRE 复现和 NPQR 设计；实验部分给出 10/20 比特主实验、30/50 比特扩展实验、消融实验和阶段耗时分析；工程部分实现网页演示、服务器接口、MCP 服务、命令行、本地测试、GitHub Pages 前端和云端部署。AI 协作部分记录了资料整理、前端开发、服

务接口、测试部署和报告写作中的使用方式，算法目标、实验口径和结果解释由作者根据代码运行结果确定。

实验结果表明,NPQR 在 10/20 比特代表样例上完成 10/10,SWAP 数均低于 SABRE basic; 选定 30/50 比特网格样例完成 4/4, 并保持 SWAP 优势。10 个主实验样例中, NPQR 合计插入 218 个 SWAP, SABRE basic 合计插入 359 个, 减少 141 个。阶段耗时显示, 不同线路的瓶颈并不相同: QAOA10 和 Brickwork20 主要受初始映射候选生成影响, QFT10 的主搜索占总耗时 90.2%。消融实验进一步说明, 束搜索和结构化初始映射对完成率和交换数都有直接影响。

项目的后续优化可以沿四个方向推进: 改进大拓扑初始映射候选生成, 压缩前沿相关候选 SWAP 集合, 引入分层路由以降低搜索状态规模, 并对模型推理做批处理。这些方向延续当前设计原则: 把学习模型放入显式、可复放的搜索框架中, 用清楚的拓扑约束和实验指标支撑量子线路路由编译的结果。

参考资料

本报告参考量子信息基础课程材料、量子路由编译论文、Qiskit 文档和项目仓库中的公开证据资料。课程材料用于支撑量子态、量子门、纠缠、噪声和量子纠错背景; 论文和文档用于说明路由算法与工程基线。

1. 钱浩亮, 浙江大学《量子信息基础》课程 PPT: 量子态、算符与矩阵、量子逻辑、量子算法、纠缠态和量子纠错相关章节。
2. G. Li, Y. Ding, and Y. Xie. Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices. *ASPLOS*, 2019. arXiv:1809.02573.
3. A. Zulehner, A. Paller, and R. Wille. An Efficient Methodology for Mapping Quantum Circuits to the IBM QX Architectures. *IEEE TCAD*, 2019.
4. A. Sinha, U. Azad, and H. Singh. Qubit Routing using Graph Neural Network aided Monte Carlo Tree Search. arXiv:2104.01992, 2021.
5. W. Tang, Y. Duan, Y. Kharkov, R. Fakoor, E. Kessler, and Y. Shi. AlphaRouter: Quantum Circuit Routing with Reinforcement Learning and Tree Search. arXiv:2410.05115, 2024.
6. Qiskit Documentation. Transpiler and SabreSwap pass documentation.
7. ZJU Quantum Compiler 项目 README、公开接口说明、示例线路、测试脚本和实验结果摘要。

A 附录：复现实验记录

本附录列出报告中使用的公开地址和复现命令。正文讨论结果和分析, 附录提供运行入口、服务启动方式和测试命令。

A.1 在线地址

GitHub:

<https://github.com/qyyqq812/ZJU-Quantum-Compiler>

GitHub Pages:

<https://qyyqq812.github.io/ZJU-Quantum-Compiler/>

A.2 本地安装

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip install -e .
qcompiler info
```

A.3 REST API

```
uvicorn src.server.app:app --host 0.0.0.0 --port 8765
curl -s http://127.0.0.1:8765/api/status
curl -s http://127.0.0.1:8765/api/examples
```

A.4 MCP 服务

```
qcompiler-mcp-http
curl -s http://127.0.0.1:8000/health
```

A.5 示例编译

```
qcompiler compile --example qft5 --backend npqr
qcompiler compile --example line_ghz50 \
    --backend sabre --topology grid_5x10
```

A.6 测试命令

```
python -m pytest -q tests/test_public_api.py \
    tests/test_public_site.py \
    tests/test_readme_contract.py \
    tests/test_submission_readiness.py
python scripts/check_submission_readiness.py
```